Software Requirements Specification

# Beacon

Location Based Attendance Tracking System

Offline-First, Self-Hostable, Open Source

| **Version** |
| --- |
| 1.2 — Draft |



| | |
| --- | --- |
| **Author** | Rudra Ponkshe |
| **Repository** | https://gitlab.com/realRudraP/beacon |
| **Licence** | To be determined |
| **Date** | April 2026 |

## Revision History

| Version | Date | Author | Description |
|---|---|---|---|
| 0.1 | 08 April 2026 | Rudra Ponkshe | Initial draft setup |
| 1.0 | 09 April 2026 | Rudra Ponkshe | First complete SRS version for TA-2 submission |
| 1.1 | 12 April 2026 | Rudra Ponkshe | Added: worker attendance history, wage calculation, remote system configuration, India public holidays, per-worker weekly off, absentia marking, petty-cash ledger, clock-in reminder notifications, short-hours warning, and under-hours dashboard flag. |
| 1.2 | 17 April 2026 | Rudra Ponkshe | Open source revision. The project is now set to natively support multi-tenant deployments, the India specific defaults are now configurable as locale-configurable records. Pluggable wage collection strategies are introduced. Push Notifications and crash reporting providers are introduced as optional and swappable. Added open source audience and reframed out of scope items as community contribution roadmap |

# Contents

**Revision History** **1**

**1 Introduction** **4**
1.1 Purpose . . . 4
1.2 Document Conventions . . . 4
1.3 Intended Audience and Reading Suggestions . . . 5
1.4 Project Scope . . . 5
1.5 References . . . 7

**2 Overall Description** **7**
2.1 Product Perspective . . . 7
2.2 Product Features . . . 8
2.3 User Classes and Characteristics . . . 10
2.4 Operating Environment . . . 10
2.5 Design and Implementation Constraints . . . 10
2.6 User Documentation . . . 11
2.7 Assumptions and Dependencies . . . 11

**3 System Features** **12**
3.1 Feature 2: Clock-In and Clock-Out . . . 13
3.2 Feature 3: Duplicate Attendance Prevention . . . 14
3.3 Feature 4: Random Mid-Day GPS Checks . . . 14
3.4 Feature 5: Manager Dashboard — Daily Attendance View . . . 15
3.5 Feature 6: Individual Staff Attendance History . . . 15
3.6 Feature 7: Staff Scheduling . . . 15
3.7 Feature 8: Holiday Management . . . 16
3.8 Feature 9: Reports and Export . . . 16
3.9 Feature 10: Attendance Edits and Overrides . . . 17
3.10 Feature 11: Worker Attendance History (Mobile) . . . 17
3.11 Feature 12: Wage Calculation Engine . . . 18
3.12 Feature 13: Remote System Configuration . . . 19
3.13 Feature 14: Holiday Management: Locale Defaults & Per-Worker Weekly Off . . . 20
3.14 Feature 15: Petty-Cash Ledger . . . 22
3.15 Feature 16: Clock-In Reminder Notifications . . . 23
3.16 Feature 17: Short-Hours Detection and Warning . . . 24

**4 External Interface Requirements** **25**
4.1 User Interfaces . . . 25
4.2 Hardware Interfaces . . . 26
4.3 Software Interfaces . . . 27
4.4 Communications Interfaces . . . 27

**5 Other Non-Functional Requirements** **28**
5.1 Performance Requirements . . . 28
5.2 Safety Requirements . . . 28
5.3 Security Requirements . . . 28

5.4 Software Quality Attributes . . . . . 29

**6 Other Requirements** **30**
6.1 Business Rules . . . . . 30
6.2 Deployment Requirements . . . . . 32

**A Glossary** **33**

**B Issues List** **35**

# 1 Introduction

## 1.1 Purpose

This Software Requirements Specification (SRS) describes the complete functional and non-functional requirements for **Beacon**, an open-source, self-hostable, location-based attendance tracking system designed for environments with sparse or no internet connectivity. The system comprises an Android mobile application for field workers and a web-based dashboard for managers and administrators.

This document serves as the single source of truth for all stakeholders, including those involved in design, development, testing, deployment, and community contribution. It is published openly alongside the source code so that contributors, deployers, and evaluators can understand the system's intent and constraints without requiring access to any private materials.

Beacon is designed to be deployed by any organisation that needs offline-capable, GPS-verified attendance tracking. While the reference deployment targets a remote farm, the system is intentionally locale-agnostic and configurable: default values reflect the reference deployment, but deployers are expected to adjust them to their operational context. All locale-specific defaults (public holidays, currency, timezone, language) are configurable without modifying source code.

## 1.2 Document Conventions

The following conventions are used throughout this document. The key words **SHALL**, **SHOULD**, and **MAY** are to be interpreted as described in RFC 2119.

| | |
|---|---|
| **FR-XX** | Functional Requirement identifier (e.g., FR-01, FR-02). |
| **NFR-XX** | Non-Functional Requirement identifier. |
| **BR-XX** | Business Rule identifier. |
| **Bold** | Used for requirement identifiers, field names, and key terms. |
| ***Italic*** | Used for priority labels and notes. |
| `Monospace` | Used for technical identifiers, API fields, and code references. |
| **SHALL** | Denotes a mandatory requirement. |
| **SHOULD** | Denotes a recommended but optional requirement. |
| **MAY** | Denotes a purely optional feature. |

Requirements are assigned one of three priority levels:

- **High** — Must be implemented in Version 1.0.
- **Medium** — Strongly recommended; may be deferred to a subsequent sprint.
- **Low** — Optional enhancement, targeted for a future community milestone.

### 1.3 Intended Audience and Reading Suggestions

Table 1: Intended Audience

| Audience | Recommended Sections |
|---|---|
| Deploying Organisation | §1, §2, §3 (feature descriptions only), §5 |
| Software Developers | All sections, especially §3 and §4 |
| QA / Testers | §3 (Acceptance Criteria), §5, Appendix B |
| System Administrators | §2.4, §4, §5.3, §6.2 |
| Open-Source Contributors | §1.4 (Contribution Roadmap), §3, §4, Appendix B (Issues List) |

### 1.4 Project Scope

Beacon is a two-component management system built for environments where internet connectivity is unreliable or intermittent. It is designed to be self-hosted: a single `docker compose up` from the repository root brings up a fully functional instance.

#### *Deployment Models*

Beacon supports the following deployment configurations:

**Single-Organisation**
: The default and simplest deployment. One instance serves one organisation. All configuration is scoped to that organisation. Suitable for a single farm, construction site, or field office.

**Multi-Tenant**
: A single backend instance hosts multiple independent organisations, each with fully isolated data, configuration, and user accounts. Intended for managed deployments or service providers offering Beacon as a hosted solution. Enabled via a compile-time flag; not required for the default self-hosted case.

#### *In Scope for Version 1.0*

- GPS geofence based clock-in and clock-out for workers through their installed Android app.
- Selfie photograph capture at clock-in for identity verification.
- Offline-first operation with background record-synchronisation upon restoring network access.
- Random mid-day GPS presence checks (frequency remotely configurable) during active work sessions.
- Worker-facing attendance history view in the Android app.
- Pluggable wage calculation engine with three built-in strategies: daily fixed rate, monthly salary divided by working days, and hourly rate. Deployers may register custom strategies.

- Remote configuration of all operational parameters: acceptable work locations (geofence), required GPS accuracy threshold, required daily work duration, and random location check frequency.
- Holiday management: a locale-configurable default holiday list (India gazetted holidays shipped as the reference preset); per-employee configurable weekly day(s) off; admin ability to mark an absence as paid or unpaid leave.
- Petty cash ledger: admin-controlled balance per worker; worker expense entry; real-time balance and transaction history visible in the Android app; all admin actions fully audited.
- Pluggable push notification provider (Firebase Cloud Messaging as the default implementation; alternative providers such as ntfy or Gotify can be substituted via configuration).
- Optional crash reporting and diagnostics provider (Firebase Crashlytics as the default; can be disabled or replaced without modifying application code).
- Daily clock-in reminder notification at a configurable time (default: 09:00 local time) on non-holiday working days.
- Post clock-out duration summary with a short-hours warning if the session was below the configured required work duration; corresponding flag in the admin dashboard.
- Web dashboard for managers and administrators: daily attendance view, holiday management, wage summary, petty-cash oversight, and report generation.
- Role-based access control (Field Worker, Manager, Admin).
- Configurable business rules: geofence tolerance, late policy, overtime, missed clock-out auto-session end.

### *Community Contribution Roadmap*

The following capabilities are out of scope for Version 1.0 but are identified as well-defined contribution opportunities for the open-source community. Each item is tracked as an issue in the project repository.

- **Payroll integration** — Automated salary disbursement or direct integration with banking or payroll APIs.
- **Inventory and bookkeeping** — Material tracking, purchase records, or general ledger features.
- **In-app messaging** — Worker-to-manager or worker-to-worker chat within the app.
- **Full HR management** — Leave applications, performance reviews, document storage, or contract management.
- **ML-based identity verification** — Facial recognition or liveness detection on selfie capture.

- **Statutory compliance calculations** — Tax deductions, provident fund (PF), employee state insurance (ESI), or TDS computation. (These are jurisdiction-specific and intentionally left to deployer-provided plugins.)
- **Additional locale presets** — Holiday calendars and wage rule presets for jurisdictions beyond India.
- **Half-day attendance marking** — Sub-day attendance granularity.
- **Alternative notification backends** — Ready-made provider implementations beyond FCM (ntfy, Gotify, APNS, etc.).

### 1.5 References

1. Beacon — Requirements Document (Draft), internal project notes, January 2026. *[Primary source for this SRS.]*
2. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*. IEEE, 1998.
3. Android Developer Documentation: Background Location Access. https://developer.android.com/training/location
4. Android Developer Documentation: WorkManager. https://developer.android.com/topic/libraries/architecture/workmanager
5. OWASP Mobile Security Testing Guide (MSTG), v1.7.0. https://owasp.org/www-project-mobile-security-testing-guide/
6. RFC 7519: JSON Web Token (JWT). IETF, 2015.
7. Ministry of Personnel, Public Grievances and Pensions, Government of India: List of Gazetted Public Holidays (annual circular). *[Shipped as the reference locale preset; not a hard dependency of the system.]*
8. Firebase Cloud Messaging (FCM) Documentation. https://firebase.google.com/docs/cloud-messaging *[Default notification provider; optional.]*

## 2 Overall Description

### 2.1 Product Perspective

Beacon is a standalone, purpose-built system and is not a module or extension of any existing platform.

The system consists of two interacting components:

1. **Android Worker App** — A lightweight native Android application installed on field workers' devices. It handles authentication, GPS capture, selfie recording, and local storage of attendance events, syncing to the backend when connectivity is available.
2. **Web Dashboard (Manager / Admin)** — A browser-based interface for management staff to monitor attendance in real time, configure schedules and policies, manage staff accounts, and export reports.

Both components communicate with a **central backend server** via a REST API over HTTPS. The Android app is designed to function independently of this connection for all core attendance actions, queuing records locally until sync is possible.

***Locale and Internationalisation***

Beacon ships with a reference configuration targeting Indian farm deployments (IST timezone, INR currency, India gazetted holidays, English UI). All locale-sensitive values are configurable without modifying source code:

- **Timezone** — configurable per organisation; all internal timestamps are stored in UTC.
- **Currency** — configurable; used for display in wage summaries and petty-cash screens.
- **Holiday presets** — the India gazetted holiday list is the default seed; administrators may replace or extend it. Additional locale presets may be contributed by the community (see §1.4 Contribution Roadmap).
- **Reminder notification time** — default 09:00 local time; configurable per organisation.

No assumption is made that deployers are located in India. The India defaults are a convenience for the reference deployment, not a requirement of the system.

### 2.2 Product Features

A high-level summary of key features is listed below. Detailed specifications appear in Section 3.

- **Geofenced Clock-in / Clock-out** with GPS accuracy validation.
- **Selfie Capture** for identity assurance at clock-in.
- **Offline-First Operation** with reliable queued sync.
- **Random Mid-Day GPS Checks** (remotely configurable frequency) for continued presence verification.
- **Duplicate Attendance Prevention** enforced on device and server.
- **Worker Attendance History** viewable on the Android app.
- **Pluggable Wage Calculation Engine** with built-in strategies: daily fixed, monthly divided, and hourly.
- **Remote System Configuration** of geofence locations, GPS accuracy threshold, required work duration, and ping frequency.
- **Holiday Management**: locale-configurable default holiday list; per-worker weekly off; absentia marking as paid/unpaid.
- **Petty-Cash Ledger** with worker expense entry, balance view, admin top-up, and full audit trail.
- **Pluggable Push Notifications** (FCM default; swappable).

- **Optional Crash Reporting** (Crashlytics default; disable-able).
- **Short-Hours Warning** on clock-out and dashboard flag.
- **Manager Dashboard** with daily view, history, anomaly flagging, and under-hours indicators.
- **Staff Scheduling** and **Holiday Management** via calendar interface.
- **Report Export** in CSV and PDF formats.
- **Attendance Editing and Overrides** with full audit trail.
- **Configurable Business Rules** for geofence, late policy, and overtime.

### 2.3 User Classes and Characteristics

Table 2: User Classes and Characteristics

| User Class | Platform | Characteristics |
|---|---|---|
| Field Worker | Android App | Field staff with basic smartphone literacy. May have limited proficiency in the deployment language. Primary concern is a simple, fast clock-in/out flow. |
| Manager | Web Dashboard | Supervisory staff who monitor daily attendance, approve anomalies, and generate reports. Moderate technical proficiency. |
| Administrator | Web Dashboard | System configurator. Sets geofence parameters, manages staff accounts and schedules. High technical proficiency expected. May be the same person as the Manager. |
| Deployer | Server / CLI | Technical operator responsible for self-hosting the system. Expected to be comfortable with Docker and basic Linux administration. Interacts with the system via `docker compose`, environment variables, and the admin dashboard. |

### 2.4 Operating Environment

**Android App** Android 10 (API level 29) or higher. Minimum 2 GB RAM. Requires camera, GPS (fine location), and background location permissions. Must operate in GPS-limited rural environments; no assumption of stable data connectivity. Google Play Services are required only when Firebase is used as the notification provider; deployments using an alternative provider do not have this dependency.

**Web Dashboard** Modern evergreen browsers: Google Chrome 110+, Mozilla Firefox 110+, Microsoft Edge 110+. Minimum screen resolution 1280 × 720. Stable broadband or mobile internet connection recommended.

**Backend Server** Linux-based server environment (Ubuntu 22.04 LTS or equivalent). Accessible via HTTPS (TLS 1.2 or higher). Must support concurrent connections from all enrolled devices and dashboard users. Distributed as a Docker image; no other runtime dependencies are required on the host.

**Network** The Android app SHALL operate fully offline. Sync is triggered automatically when any network interface (WiFi or mobile data) becomes available. No minimum bandwidth is assumed for field devices.

### 2.5 Design and Implementation Constraints

1. The Android application SHALL be developed natively for Android (Kotlin); cross-platform frameworks are discouraged due to GPS and background task reliability

requirements.

2. All data transmission between client and server SHALL use HTTPS with TLS 1.2 or higher.
3. Selfie images SHALL be compressed before transmission to minimise bandwidth consumption (target: under 200 KB per image).
4. The system SHALL NOT transmit raw GPS coordinates to any third-party analytics service.
5. Background GPS ping frequency SHALL be configurable and SHALL NOT exceed 4 pings per active work session to preserve battery life.
6. All attendance records SHALL store timestamps in UTC; display conversion to local time is the responsibility of the client layer.
7. The web dashboard SHALL be accessible without installing any browser plugin or extension.
8. Push notification and crash reporting integrations SHALL be implemented behind provider interfaces. The default implementations use Firebase; these MAY be replaced by alternative implementations via environment variable configuration. The core application SHALL NOT contain unconditional references to any specific provider SDK.
9. The wage calculation engine SHALL be implemented as a set of named, selectable strategies. Built-in strategies are described in FR-13. Custom strategies MAY be registered by deployers without modifying core application code.

### 2.6 User Documentation

The following documentation SHALL be delivered alongside the system:

- **Field Worker Quick-Start Card**: A single-page visual guide (printable) covering app installation, login, clock-in, and clock-out.
- **Manager / Admin User Manual**: A full guide covering all dashboard functions: attendance monitoring, scheduling, holiday management, exports, and system configuration.
- **Self-Hosting Guide**: Covers Docker-based deployment, environment variable reference, backup procedures, and troubleshooting. Targeted at deployers.
- **Contributor Guide**: Covers repository structure, development environment setup, architecture decision records, and contribution workflow. Published in the repository root as `CONTRIBUTING.md`.
- **In-App Help**: Contextual error messages on the Android app SHALL be clear and actionable (see FR-02 acceptance criteria).

### 2.7 Assumptions and Dependencies

**Assumptions:**

1. Each field worker is issued or personally owns an Android device meeting the minimum hardware specification.
2. The work premises have a definable GPS boundary suitable for geofence configuration.
3. The deploying organisation will provide accurate GPS coordinates for all acceptable work locations prior to system configuration.
4. Workers will carry their assigned device during all working hours.
5. At least one manager or administrator will have reliable internet access to operate the web dashboard.
6. The deploying organisation will select an appropriate wage calculation strategy and configure it before the system goes live.
7. The default holiday list (India gazetted holidays) will be reviewed and adjusted by the administrator to match the deployment locale before going live.
8. Petty-cash balances are managed exclusively by the admin; workers can only add expense records, not modify balances directly.

**Dependencies:**

1. When Firebase is used as the notification provider, Google Play Services must be available on all worker devices. This dependency is lifted when an alternative notification provider is configured.
2. The backend server must be deployed and reachable over the internet before the Android app can perform any sync operation.
3. A push notification infrastructure endpoint (FCM or equivalent) is required for admin notifications, and optionally worker notifications in the future.
4. An accurate system clock (NTP-synchronised) is assumed on the backend server for all timestamp generation.

## 3 System Features

**FR-01: Employee Authentication** *Priority:*

**Description:**

The system SHALL allow a field worker to log in using a username and password assigned by the administrator. Each device SHALL be associated with exactly one employee account, and the device ID and model SHALL be recorded for audit purposes.

**Acceptance Criteria:**

- Login succeeds only for accounts with valid, active credentials.
- On successful login, the device ID is recorded server-side.
- The device shall remain logged in even when it's offline for a set number of days, which SHALL be defined in the configuration, following which it has to re-authenticate with the server.

## 3.1 Feature 2: Clock-In and Clock-Out

***Description and Priority***

Core attendance recording functionality. Workers record the start and end of their work session via the Android app with GPS and photo verification. *Priority: High*

***Functional Requirements***

**FR-02: Clock-In (Attendance Start)** *Priority:*

**Description:**

The system SHALL allow a logged-in field worker to initiate a clock-in event for the current working day, subject to geofence and GPS accuracy validation.

**Rules:**

- The worker SHALL be within the configured work premises geofence boundary (within the allowed tolerance).
- A selfie photograph SHALL be captured at the moment of clock-in.
- Clock-in SHALL function without an active internet connection.

**Captured Data:**

- `timestamp` (UTC)
- `gps_lat`, `gps_lon` (decimal degrees)
- `gps_accuracy` (metres)
- `selfie_image_url` (populated after sync)
- `device_id`, `app_version`

**Acceptance Criteria:**

- Clock-in succeeds only if the worker is inside the geofence AND GPS accuracy is within the configured threshold.
- If offline, the record is stored in local device storage and automatically synced upon network restoration.
- On failure, the app SHALL display the specific reason (e.g., "Outside geofence", "GPS accuracy too low", "Camera permission denied").

**FR-03: Clock-Out (Attendance End)** *Priority:*

**Description:**

The system SHALL allow a worker with an active clock-in record to record the end of their work session.

**Rules:**

- A clock-out SHALL only be possible if a clock-in exists for the current day.

- GPS coordinates SHALL be captured at clock-out.
- Selfie capture at clock-out is configurable (see open issues, Appendix B).
- Clock-out location enforcement (geofence) is configurable.

**Acceptance Criteria:**

- The clock-out button SHALL only be visible when an active clock-in record exists for the day.
- Offline clock-out is stored locally and synced upon network restoration.

### 3.2 Feature 3: Duplicate Attendance Prevention

***Description and Priority***

The system prevents workers from clocking in or out more than once per day without managerial override. *Priority: High*

***Functional Requirements***

**FR-04: Prevent Duplicate Attendance** *Priority:*

**Acceptance Criteria:**

- Once a clock-in record exists for a given date, further clock-ins on that date SHALL be blocked unless a manager explicitly overrides.
- Clock-out SHALL be permitted only once per clock-in record.
- Duplicate prevention SHALL be enforced both on-device and server-side (to handle sync edge cases).

### 3.3 Feature 4: Random Mid-Day GPS Checks

***Description and Priority***

The system performs randomized background GPS pings during an active work session to provide continued presence assurance without requiring manual action from the worker. *Priority: Medium*

***Functional Requirements***

**FR-05: Random Mid-Day GPS Checks** *Priority:*

**Rules:**

- Pings SHALL only occur during an active work session (between clock-in and clock-out).
- Ping frequency SHALL be configurable (e.g., 1–4 pings per day).
- Each ping records: GPS coordinates, GPS accuracy, and timestamp.
- Ping behaviour SHALL be designed to minimise battery drain.

**Acceptance Criteria:**

- GPS pings are queued offline and synced upon connectivity.
- If background location permissions or OS restrictions prevent pings, the system SHALL log the reason server-side.

### 3.4 Feature 5: Manager Dashboard — Daily Attendance View

***Description and Priority***

Managers can view a consolidated daily attendance summary for all staff, including flags for anomalies. *Priority: High*

***Functional Requirements***

**FR-06: Daily Attendance View** *Priority:*

**Description:**

The web dashboard SHALL present a daily attendance list showing, for each employee: clock-in time, clock-out time, total hours worked, and any flags or issues (e.g., outside geofence, low GPS accuracy, missed clock-out).

**Acceptance Criteria:**

- The attendance list SHALL load within 2 seconds for up to 500 employee records on a standard broadband connection.
- The list SHALL be filterable by date, employee name, and flag status.

### 3.5 Feature 6: Individual Staff Attendance History

***Description and Priority***

Managers can drill down into a per-employee attendance history view. *Priority: High*

***Functional Requirements***

**FR-07: Staff Attendance History** *Priority:*

**Description:**

The dashboard SHALL allow managers to view a selected employee's attendance history, including daily records, total hours, anomaly flags, and selfie photo references (subject to privacy configuration).

### 3.6 Feature 7: Staff Scheduling

***Description and Priority***

Administrators and managers can configure working day patterns per employee. *Priority: Medium*

***Functional Requirements***

**FR-08: Staff Scheduling (Working Days per Employee)** *Priority:*

**Rules:**

- Each employee SHALL have a configurable working day pattern (Monday–Sunday toggles or a shift template).
- Schedule changes SHALL apply to future dates by default; retroactive application is configurable.

**Acceptance Criteria:**

- Schedule changes are immediately reflected in the attendance dashboard view.
- If a worker clocks in on a non-working day, the record is flagged automatically.

### 3.7 Feature 8: Holiday Management

***Description and Priority***

Administrators can define public and organisation-specific holidays via a calendar interface. *Priority:* *Medium*

***Functional Requirements***

**FR-09: Public Holiday Management** *Priority:*

**Rules:**

- Holidays SHALL be markable on a calendar interface.
- Each holiday SHALL be configurable as paid or unpaid.
- Holidays SHALL be applicable to all employees or a selected subset.

**Acceptance Criteria:**

- Holidays are visible in the dashboard calendar view.
- Attendance recorded on a holiday is flagged or handled according to the configured rule.

### 3.8 Feature 9: Reports and Export

***Description and Priority***

Managers can generate and export attendance reports for payroll and compliance purposes. *Priority:* *High*

***Functional Requirements***

**FR-10: Attendance Report Export** *Priority:*

**Rules:**

- Reports SHALL be exportable in CSV and PDF formats.
- Export SHALL be filterable by date range and/or individual employee.
- Reports SHALL include: days present, days absent, total hours, and overtime hours.

### 3.9 Feature 10: Attendance Edits and Overrides

***Description and Priority***

Managers or administrators can correct erroneous records or add missing entries, with a complete audit trail. *Priority: Medium*

***Functional Requirements***

**FR-11: Attendance Edits and Overrides** *Priority:*

**Rules:**

- Authorised users SHALL be able to add missing entries, correct existing entries, and approve flagged anomalies.
- Every edit SHALL store: editor identity, edit timestamp, and a mandatory reason note.
- The original record SHALL be preserved for audit; edits create a versioned history.

### 3.10 Feature 11: Worker Attendance History (Mobile)

***Description and Priority***

Field workers can view their own attendance history directly within the Android app, without requiring access to the web dashboard. *Priority: High*

***Functional Requirements***

**FR-12: Worker Self-Service Attendance History** *Priority:*

**Description:**

The Android app SHALL provide a dedicated screen showing the authenticated worker's attendance records, including date, clock-in time, clock-out time, total hours worked, holiday status, and any flags (e.g., short hours, outside geofence).

**Rules:**

- History SHALL be browsable by month with a summary of days present, days absent, and total hours.
- Records SHALL be read-only for the worker; no editing is permitted from this view.

- History SHALL be available offline for previously synced records.

**Acceptance Criteria:**

- The attendance history screen loads within 2 seconds for up to 90 days of records from local cache.
- Workers can only view their own records; cross-worker data access is not permitted.

### 3.11 Feature 12: Wage Calculation Engine

***Description and Priority***

The system automatically calculates each worker's earned wages for a given period using a configurable, pluggable calculation strategy. *Priority: High*

***Functional Requirements***

**FR-13: Pluggable Wage Calculation Engine** *Priority:*

**Description:**

The system SHALL calculate a worker's gross wages for any date range using the strategy configured for that worker. Each worker SHALL have a `wage_strategy` field that selects which calculation strategy to apply. Three strategies are built in; deployers MAY register additional strategies without modifying core application code.

**Built-in Wage Strategies:**

| | |
|---|---|
| `DAILY_FIXED` | *(Default)* The worker has a configured daily rate. Wage = paid days × daily rate. A day is a "paid day" if the worker attended, or if the day is a paid public holiday, or if the admin has marked the absence as paid leave. Weekly off days and unpaid absences are excluded. |
| `MONTHLY_DIVIDED` | The worker has a configured monthly salary. For a given month, the effective daily rate = monthly salary ÷ number of calendar days in that month. Wage = effective daily rate × paid days. Useful when the employment contract specifies a fixed monthly figure rather than a per-day rate. |
| `HOURLY` | The worker has a configured hourly rate. Wage = hourly rate × total hours worked, derived from session durations in the attendance records. Overtime hours (if the overtime policy is enabled) MAY be calculated at a separate configured multiplier. |

**Rules (all strategies):**

- Weekly off days are excluded from all wage calculations regardless of strategy.
- Wage summaries SHALL be viewable by the worker on the Android app and by managers/admins on the dashboard.
- The wage calculation SHALL be performed server-side; client-side figures are display-only and refreshed on each sync.

**Acceptance Criteria:**

- Wage calculations update automatically when attendance records, leave markings, or rate/salary values are modified.
- The Android app displays the worker's current month wage summary on the home screen or a dedicated earnings screen.
- Managers can export wage summaries per employee or for all employees for a selected date range (CSV / PDF).
- Switching a worker's strategy recalculates wages from the effective date of the change; historical records are not retroactively recalculated unless explicitly triggered by an admin.

### 3.12 Feature 13: Remote System Configuration

***Description and Priority***

Administrators can remotely configure all operational parameters of the system from the web dashboard. Changes propagate to worker devices on next sync without requiring an app update. *Priority: High*

***Functional Requirements***

**FR-14: Remote Configuration Panel** *Priority:*

**Description:**

The admin dashboard SHALL provide a configuration panel through which all system-wide and per-worker operational parameters can be updated remotely.

**Configurable Parameters:**

- **Acceptable Work Locations:** One or more named geofence zones (radius or polygon), each with a centre-point, radius, and boundary tolerance. Multiple zones may be active simultaneously.
- **Required GPS Accuracy Threshold:** Maximum acceptable GPS error radius (metres) for clock-in acceptance.
- **Required Daily Work Duration:** Minimum hours per day that constitute a full attendance; used for short-hours detection.
- **Random Location Check Frequency:** Number of background GPS pings per active work session (range: 0–4).
- **Offline Login Window:** Maximum duration (days) for cached credential login without server re-authentication.
- **Missed Clock-Out Auto-Close:** Enable/disable and threshold in hours.
- **Late Clock-In Policy:** Threshold time and grace period.
- **Overtime Policy:** Daily threshold and rounding rule.

- **Wage Strategy (per worker):** One of `DAILY_FIXED`, `MONTHLY_DIVIDED`, `HOURLY`, or a custom registered strategy identifier.
- **Notification Provider:** Identifier of the active push notification provider (e.g., `fcm`, `ntfy`). Read-only on the dashboard; set via server environment configuration.

**Rules:**

- Configuration changes SHALL be versioned and timestamped.
- The Android app SHALL fetch the latest configuration on every successful sync and apply it immediately.
- Configuration is applied system-wide unless a parameter supports per-worker override (noted per parameter above).
- All configuration changes SHALL be recorded in the audit log with the admin identity.

**Acceptance Criteria:**

- A configuration change made on the dashboard is reflected on a connected worker device within one sync cycle (target: 60 seconds on available network).
- Invalid configuration values (e.g., negative GPS threshold) are rejected with a descriptive error message.

### 3.13 Feature 14: Holiday Management: Locale Defaults & Per-Worker Weekly Off

***Description and Priority***

The system ships with a locale-configurable default holiday list. The reference preset contains all Government of India gazetted public holidays and is applied by default, but administrators are expected to review and adjust it to match their deployment context. Administrators can additionally configure per-employee weekly off days and ad-hoc holidays. *Priority: High*

***Functional Requirements***

**FR-15: Locale-Configurable Holiday Defaults** *Priority:*

**Description:**

The system SHALL ship with a pre-seeded holiday list for the active locale preset. The reference preset (India) contains all Government of India gazetted national and festival public holidays for the current calendar year, marked as paid by default.

**Rules:**

- The active locale preset SHALL be configurable via the admin dashboard or server environment without a software release.
- The reference India preset SHALL include at minimum: Republic Day (26 Jan), Holi, Good Friday, Ambedkar Jayanti (14 Apr), Labour Day (1 May), Independence

Day (15 Aug), Gandhi Jayanti (2 Oct), Dussehra, Diwali, and Christmas Day (25 Dec).

- Administrators SHALL be able to add, remove, or reclassify any holiday (paid ↔ unpaid) for their organisation.
- Holiday applicability can be set to: all employees, or a named subset.
- Additional locale presets (holiday lists for other jurisdictions) MAY be contributed to the repository and selected via configuration.

**Acceptance Criteria:**

- On a configured paid holiday, a worker who does not clock in SHALL NOT be flagged as absent; the day is recorded as a paid holiday automatically.
- The holiday calendar is visible to workers on the Android app (read-only).

### FR-16: Per-Worker Weekly Off Configuration *Priority:*

**Description:**

Each worker's weekly day(s) off SHALL be individually configurable by the administrator. Weekly off days are excluded from attendance expectations and wage calculations.

**Rules:**

- Default weekly off: Sunday (configurable to any day or combination of days).
- Changes to a worker's weekly off pattern apply from the effective date forward; historical records are not retroactively modified.
- If a public holiday falls on a weekly off day, the holiday takes precedence in display; no additional compensation is implied by this SRS.

**Acceptance Criteria:**

- A worker's weekly off day does not trigger an absent flag or a clock-in reminder notification.
- Weekly off days are visually distinguished in the worker's attendance history view.

### FR-17: Absentia Marking as Paid / Unpaid Leave *Priority:*

**Description:**

When a worker is absent on a working day (not a public holiday or weekly off), the administrator SHALL be able to retroactively mark the absence as either paid leave or unpaid leave.

**Rules:**

- Absent days with no marking default to *unpaid*.
- A paid leave marking causes the day to count as a paid day in wage calculations.
- An unpaid leave marking explicitly records the day as excluded from wages.

- All leave markings SHALL be recorded in the audit log with admin identity, timestamp, and optional note.
- The worker's attendance history view in the Android app SHALL reflect the leave classification (paid leave / unpaid leave / absent) once synced.

**Acceptance Criteria:**

- Leave marking updates wage calculations immediately upon save.
- The worker can see the leave classification in their attendance history; they cannot modify it.

### 3.14 Feature 15: Petty-Cash Ledger

***Description and Priority***

The system maintains a per-worker petty-cash balance. Workers can record individual expenditures; administrators control the balance and can view all transactions. All admin actions are audited and visible as entries in the worker's transaction feed. *Priority: High*

***Functional Requirements***

**FR-18: Worker Petty-Cash View and Expense Entry** *Priority:*

**Description:**

The Android app SHALL display the worker's current petty-cash balance and a chronological list of all transactions (credits and debits), including admin-initiated entries.

**Rules:**

- Workers can create *expense records* (debit entries) containing: amount, date, and a short description.
- Workers can view their current balance and full transaction history.
- Workers CANNOT modify or delete existing entries.
- Admin-initiated entries (top-ups, adjustments, corrections) SHALL appear in the worker's transaction feed with a system label (e.g., "Admin credit — [note]").
- The petty-cash screen SHALL function offline; expense records are queued and synced.

**Captured Data per Expense Entry:**

- `entry_id`, `worker_id`
- `entry_type`: `debit` (worker expense) or `credit` (admin top-up / adjustment)
- `amount` (positive decimal)
- `description` (free text, max 200 characters)
- `entry_date` (worker-entered date; defaults to today)
- `created_at` (UTC timestamp of record creation)

- `created_by`: worker ID or admin ID
- `audit_note` (mandatory for admin-initiated entries)

**Acceptance Criteria:**

- The balance displayed is the sum of all credits minus all debits for the worker.
- New worker expense entries appear immediately in the local transaction list; they are marked as "pending sync" until server acknowledgement.
- Admin entries appear in the worker's feed within one sync cycle of the admin action.

**FR-19: Admin Petty-Cash Management** *Priority:*

**Description:**

Administrators SHALL be able to view all workers' petty-cash balances and transaction histories, and to update any worker's balance via credit or adjustment entries, each with a mandatory audit note.

**Rules:**

- The dashboard SHALL show a petty-cash summary table: all workers, current balance, last transaction date.
- Admin can drill into any worker's full transaction history.
- Admin can add a credit entry (top-up) or a corrective adjustment (positive or negative) with a mandatory note.
- Admin CANNOT delete worker expense entries; corrections must be made via adjustment entries.
- Every admin action (top-up, adjustment, balance reset) is recorded in the system-wide audit log AND creates a visible entry in the affected worker's transaction feed.

**Acceptance Criteria:**

- An admin credit entry updates the worker's displayed balance within one sync cycle on the worker's device.
- The audit log entry for every admin petty-cash action includes: admin identity, target worker, entry type, amount, note, and UTC timestamp.
- The petty-cash dashboard view loads within 2 seconds for up to 500 workers.

### 3.15 Feature 16: Clock-In Reminder Notifications

***Description and Priority***

The Android app notifies workers to mark their attendance on working days where they have not yet clocked in. Notification delivery is handled by a pluggable provider; Firebase Cloud Messaging is the default. *Priority: Medium*

***Functional Requirements***

**FR-20: Daily Clock-In Reminder** *Priority:*

**Description:**

The system SHALL send a push notification to a worker's device at the configured reminder time (default: 09:00 local time) on any day that is a scheduled working day (not a public holiday, not the worker's weekly off day) if the worker has not yet clocked in for that day.

**Rules:**

- Notifications SHALL be delivered via the configured push notification provider. The provider is selected via server environment configuration; the application SHALL NOT hardcode a provider.
- The notification SHALL NOT be sent if the worker has already clocked in for the day by the reminder time.
- The notification SHALL NOT be sent on public holidays or the worker's configured weekly off day(s).
- If the device is offline at the scheduled time, the notification SHALL be delivered as soon as the device comes online, provided the worker has still not clocked in for that day.
- Workers SHALL be able to disable this notification in the app settings (opt-out).
- The reminder time SHALL be configurable per organisation (default: 09:00 local time).

**Acceptance Criteria:**

- Notification is not delivered on holidays or weekly off days.
- Notification is not delivered if clock-in already exists.
- Tapping the notification opens the clock-in screen directly.

### 3.16 Feature 17: Short-Hours Detection and Warning

***Description and Priority***

Upon clock-out, the app informs the worker of their total session duration and warns them if they have worked less than the required daily duration. The same flag appears in the manager dashboard. *Priority: Medium*

***Functional Requirements***

**FR-21: Post-Clock-Out Duration Summary and Warning** *Priority:*

**Description:**

Immediately after a successful clock-out, the Android app SHALL display a summary screen showing the total duration of the work session. If the duration is less than the

configured required work duration (see FR-14, BR-09), a short-hours warning SHALL be prominently displayed.

**Rules:**

- Session duration = clock-out timestamp − clock-in timestamp.
- The warning is informational only; no automated action (deduction, lock-out, or escalation) is triggered.
- The short-hours flag SHALL be recorded on the attendance record server-side.
- The manager dashboard daily attendance view SHALL display a visible indicator (e.g., flag or colour coding) on any record where the session duration was below the required threshold.
- The flag is visible to managers; no notification is sent to the manager automatically.

**Acceptance Criteria:**

- The post-clock-out screen always shows total session duration in hours and minutes.
- If duration < configured minimum, a warning message is shown before the worker can dismiss the screen.
- The short-hours flag is visible in the manager dashboard for the affected date.
- If the session was auto-closed (missed clock-out), the flag applies if the auto-close duration is less than the minimum.

# 4 External Interface Requirements

## 4.1 User Interfaces

**Android Worker App screens:**

- **Login Screen**: Username/password fields.
- **Home Screen**: Today's attendance status (clocked in / not clocked in / clocked out), current month wage summary, petty-cash balance snapshot, and sync status indicator.
- **Clock-In Flow**: GPS acquisition indicator, selfie camera capture, confirmation, and error message display.
- **Clock-Out Flow**: GPS confirmation, optional selfie, post-clock-out duration summary screen with short-hours warning if applicable.
- **Attendance History Screen**: Monthly calendar / list view of the worker's own attendance records; read-only.
- **Earnings Screen**: Month-to-date wage calculation breakdown (days attended or hours worked depending on strategy, rate, total earned).
- **Petty-Cash Screen**: Current balance, chronological transaction list (worker expenses and admin entries), and add-expense form.

- **Holiday Calendar Screen**: Read-only view of organisation holidays and the worker's weekly off days.
- **Sync Status Screen**: Queue of pending records, last sync time, retry option.
- **Settings Screen**: Notification opt-out toggle and app information.

**Web Dashboard screens:**

- **Login**: Credential entry with session management.
- **Daily Attendance View**: Tabular summary with filters, flag indicators, and short-hours markers.
- **Employee List**: Searchable, paginated list with account management actions.
- **Employee History**: Per-employee timeline view with leave classification, wage strategy, and wage summary.
- **Absentia / Leave Marking**: Interface to mark an absent day as paid or unpaid leave for a specific employee.
- **Wage Summary**: Per-employee and aggregate wage calculation view with strategy label; exportable.
- **Scheduling Calendar**: Visual week/month calendar with working day toggles per employee.
- **Holiday Calendar**: Date picker for marking holidays with type (paid/unpaid) and applicability settings; active locale preset holidays visible and editable.
- **Petty-Cash Dashboard**: Summary table of all worker balances; drill-down to individual transaction history; add credit / adjustment form.
- **Export / Reports**: Date range selector, format selector (CSV / PDF), download trigger; covers attendance, wages, and petty-cash.
- **Admin Settings / Remote Configuration**: All operational parameters: geofence zones, GPS accuracy threshold, required work duration, ping frequency, login window, policy toggles, wage strategies, and locale settings.
- **Audit Log Viewer**: Searchable, filterable log of all admin actions across attendance, configuration, and petty-cash.

UI design SHALL prioritise simplicity and minimal tap-count for the worker app. Error messages SHALL be written in plain language, not technical codes.

### 4.2 Hardware Interfaces

**GPS Receiver** The Android app SHALL interface with the device GPS receiver via the Android Location API (FusedLocationProviderClient). Accuracy metadata SHALL be recorded with every GPS reading.

**Camera** The app SHALL access the device camera via Android Camera2 API or CameraX for selfie capture. Front or rear camera selection is configurable (see Appendix B, issue 5).

**Storage** Local SQLite or Room database on the Android device SHALL be used to persist offline attendance records.

**Server Hardware**
No specific server hardware is mandated; the backend SHALL run on any Linux-based virtual machine or cloud instance meeting minimum CPU and memory specifications.

### 4.3 Software Interfaces

**Android SDK**
Minimum API level 29 (Android 10). Google Play Services required only when Firebase notification provider is active.

**REST API** The Android app and web dashboard communicate with the backend via a RESTful API over HTTPS. Request and response bodies use JSON format (UTF-8 encoding).

**Authentication**
JSON Web Tokens (JWT, RFC 7519) SHALL be used for session management. Tokens SHALL have a configurable expiry.

**Database** Backend database technology is not mandated by this SRS but SHALL support ACID-compliant transactions given the audit requirements.

**Notification Provider Interface**
The backend SHALL expose a `NotificationProvider` interface. The FCM implementation is bundled by default. An alternative implementation MAY be activated by setting the `BEACON_NOTIFICATION_PROVIDER` environment variable and providing the corresponding configuration.

**Crash Reporting Interface**
The Android app SHALL expose a `CrashReporter` interface. The Firebase Crashlytics implementation is bundled by default and MAY be disabled by setting `BEACON_CRASH_REPORTING=disabled`. When disabled, the no-op implementation is activated and no data leaves the device.

**Export Libraries**
Server-side libraries for PDF generation and CSV serialisation are required for FR-10.

### 4.4 Communications Interfaces

**HTTPS / TLS**
All client–server communication SHALL use HTTPS with TLS 1.2 or higher. Self-signed certificates SHALL NOT be used in production.

**Sync Protocol**
The Android app SHALL use a background sync mechanism (WorkManager or equivalent) that retries failed sync attempts with exponential backoff, up to a maximum retry count.

**Image Upload**

Selfie images SHALL be uploaded as multipart/form-data or as Base64-encoded payloads within the JSON body, with a maximum compressed size of 200 KB per image.

**Push Notifications**

Push notifications SHALL be dispatched via the configured `NotificationProvider`. The default provider is Firebase Cloud Messaging (FCM). Provider selection does not require an app update; it is a server-side configuration change.

# 5 Other Non-Functional Requirements

## 5.1 Performance Requirements

**NFR-01** **App Startup:** The Android app SHALL reach the home screen within 3 seconds on a mid-range device (2 GB RAM, Android 10, cold start).

**NFR-02** **Clock-In Response:** The clock-in confirmation SHALL be displayed to the user within 5 seconds of tapping the clock-in button, assuming GPS is already acquired.

**NFR-03** **Sync Latency:** Offline-queued records SHALL sync to the server within 30 seconds of network restoration on a connection of at least 100 kbps (2G equivalent).

**NFR-04** **Dashboard Load:** The daily attendance view SHALL render within 2 seconds for up to 500 employee records on a standard broadband connection ($\geq$ 10 Mbps).

**NFR-05** **Battery Impact:** Background GPS ping operations SHALL NOT cause measurable battery drain exceeding 5% per hour above the device idle baseline.

## 5.2 Safety Requirements

**NFR-06** **Data Loss Prevention:** No attendance record SHALL be permanently lost due to a sync failure. Records SHALL remain in local storage until server-side acknowledgement is received.

**NFR-07** **Graceful Degradation:** If the backend server is unavailable, the Android app SHALL continue to accept clock-in and clock-out events without displaying an error to the worker (beyond a sync-pending indicator).

**NFR-08** **Conflict Resolution:** In the event of a sync conflict (e.g., manager edited a record while an offline record exists), the system SHALL flag the conflict for manual review rather than silently overwriting either version.

## 5.3 Security Requirements

**NFR-09** **Authentication:** All API endpoints SHALL require a valid JWT token. Unauthenticated requests SHALL return HTTP 401.

**NFR-10** **Role-Based Access Control:** The system SHALL enforce role boundaries between Field Worker, Manager, and Admin. A field worker SHALL NOT be able to access any manager or admin endpoint.

**NFR-11** **Audit Log:** All attendance edits, overrides, and administrative actions (account creation/deletion, config changes) SHALL be recorded in an immutable audit log with actor identity and timestamp.

**NFR-12** **Data in Transit:** All data transmitted between any client and the server SHALL be encrypted using TLS 1.2 or higher.

**NFR-13** **Data at Rest:** Selfie images and personal data stored on the server SHALL be stored with access controls limiting retrieval to authorised roles.

**NFR-14** **Device Binding:** The system SHALL record device ID at login and flag anomalies where the same account logs in from a different device without prior de-registration.

**NFR-15** **Petty-Cash Ledger Integrity:** The petty-cash balance SHALL be calculated server-side as the authoritative sum of all credit and debit entries. Client-side balance is for display only and is overwritten on each sync. No balance field SHALL be stored as a mutable scalar to prevent tampering.

**NFR-16** **Wage Calculation Consistency:** Wage figures displayed on the worker app and manager dashboard SHALL be derived from the same server-side calculation engine. Discrepancies between views are not permitted.

**NFR-17** **Notification Delivery:** Clock-in reminder notifications SHALL be delivered within 5 minutes of the scheduled reminder time on a device with active network connectivity.

**NFR-18** **Configuration Propagation:** Remote configuration changes SHALL reach all active devices within 60 seconds of save on a device with network connectivity.

**NFR-19** **Provider Isolation:** The crash reporting provider SHALL NOT have access to attendance records, GPS coordinates, selfie images, wage data, or petty-cash data. It SHALL receive only anonymised crash context (stack traces, device model, OS version, app version).

### 5.4 Software Quality Attributes

**Reliability** The backend service SHALL target 99.5% uptime (excluding scheduled maintenance windows). The Android app SHALL have zero data loss under all network conditions.

**Maintainability** The codebase SHALL follow a documented coding standard. All business rule thresholds (geofence radius, ping frequency, late-in time, etc.) SHALL be stored as configurable parameters, not hard-coded values. Provider interfaces SHALL be documented with enough detail for a contributor to implement a new provider without reading core application code.

**Usability** The full clock-in flow (app open to confirmation) SHALL be completable in under 60 seconds by a first-time user following the quick-start card.

**Portability** The web dashboard SHALL function correctly on Chrome, Firefox, and Edge (current and previous major versions) without any browser-specific workarounds.

**Debuggability**
The Android app SHALL maintain an on-device event log ("black box") exportable by administrators for support purposes. The backend SHALL assign correlation IDs to all requests for end-to-end tracing.

**Scalability** The system architecture SHALL support scaling to at least 1,000 concurrent active devices without architectural changes.

**Self-Hostability**
A deployer with basic Docker and Linux knowledge SHALL be able to bring up a fully functional instance by following the self-hosting guide, without writing any code or modifying any source files.

# 6 Other Requirements

## 6.1 Business Rules

The following rules govern system behaviour. All default values are appropriate for the reference deployment (remote farm, India) and are expected to be reviewed and adjusted by the deploying organisation. Every rule marked as *configurable* can be changed via the admin dashboard or server environment without modifying source code.

**BR-01 Geofence Type**
Geofence boundary may be defined as a radius (circular) or a polygon. Multiple named zones may be active simultaneously. Default: single radius zone. *Configurable.*

**BR-02 Geofence Tolerance**
Acceptable distance outside the geofence boundary before clock-in is rejected. Default: 150 m. *Configurable.*

**BR-03 GPS Accuracy Threshold**
Maximum acceptable GPS error radius for clock-in. Default: 50 m. *Configurable* (FR-14).

**BR-04 Clock-Out Location**
Whether clock-out is permitted outside the geofence. Default: Yes (allowed anywhere). *Configurable.*

**BR-05 Missed Clock-Out**
Whether to auto-close a session after a configurable number of hours. Default: Yes, after 10 hours. Manager notification on auto-close: Yes. *Configurable.*

**BR-06 Late Policy**
A clock-in after a configurable time is flagged as late. Grace period

configurable (default: 15 minutes). Optional; disabled by default. *Configurable.*

**BR-07 Overtime Policy**
Hours worked beyond a daily threshold are flagged as overtime. Rounding rule configurable (e.g., nearest 15 minutes). Optional; disabled by default. *Configurable.*

**BR-08 Mid-Day Ping Frequency**
Number of random GPS checks per active session. Range: 0–4. Default: 2. *Configurable* (FR-14).

**BR-09 Required Daily Work Duration**
Minimum session duration constituting a full day's work. Used for short-hours detection (FR-21). Default: 8 hours. *Configurable* (FR-14).

**BR-10 Worker Weekly Off Day**
Default weekly off day for all workers: Sunday. Overridable per worker to any day(s) of the week (FR-16). Weekly off days are excluded from attendance expectations and wage calculations. *Configurable per worker.*

**BR-11 Holiday Locale Preset**
At system initialisation, the active locale preset is seeded as the organisation's default paid holiday list (FR-15). The reference preset is the India gazetted holiday list. Deployers in other locales should replace or extend this list before going live. *Configurable without a software release.*

**BR-12 Absent Day Default Classification**
An absent working day with no admin leave marking defaults to *unpaid*. Admins may reclassify to paid or unpaid leave (FR-17). *Configurable.*

**BR-13 Wage Strategy and Rate**
Each worker has an individually configurable wage strategy (`DAILY_FIXED`, `MONTHLY_DIVIDED`, `HOURLY`, or a custom strategy) and the corresponding rate or salary figure. Currency is configurable per organisation; default: INR. *Configurable per worker.*

**BR-14 Clock-In Reminder Time**
Daily reminder notification time. Default: 09:00 local time. *Configurable per organisation* (FR-14).

**BR-15 Petty-Cash Edit Policy**
Workers may add expense (debit) entries only. Admins may add credit or adjustment entries. No entry may be deleted by any user; corrections are made via adjustment entries (FR-19). *Not overridable; this is a data integrity rule.*

### 6.2 Deployment Requirements

1. The system SHALL be distributable as a set of Docker images orchestrated by a `docker-compose.yml` in the repository root. A deployer SHALL be able to bring up a fully functional instance with a single command.
2. An `.env.example` file SHALL be provided listing all required and optional environment variables with descriptions and default values.
3. The system SHALL undergo a field trial on a minimum of 1 device for 2–3 days before any production rollout.
4. Full rollout SHALL proceed only after the field trial produces no critical defects.
5. An installation guide SHALL be provided for sideloading the APK onto worker devices (Google Play distribution is not assumed).

# A Glossary

**API** Application Programming Interface. The defined set of endpoints through which the Android app and web dashboard communicate with the backend server.

**APK** Android Package Kit. The installation file format for Android applications.

**Audit Log** An immutable, chronological record of all privileged actions performed by administrators, including attendance edits, leave markings, petty-cash adjustments, and configuration changes.

**Clock-In** The action of recording the start of a field worker's attendance session, captured with GPS coordinates and a selfie photograph.

**Clock-Out** The action of recording the end of a field worker's attendance session.

**CrashReporter** The interface through which the Android app reports unhandled exceptions and diagnostics. Firebase Crashlytics is the default implementation; it may be replaced or disabled via configuration.

**Daily Rate** The configured gross wage amount earned by a worker for one full paid day of attendance. Used by the `DAILY_FIXED` wage strategy. Currency is configurable per organisation.

**Deployer** A technical operator who installs and maintains a self-hosted Beacon instance on behalf of an organisation.

**FCM** Firebase Cloud Messaging. Google's cross-platform push notification service. The default `NotificationProvider` implementation.

**Gazetted Holiday** A public holiday officially notified in a government gazette. The reference locale preset uses the Government of India gazette.

**Geofence** A virtual geographic boundary defined by GPS coordinates (radius or polygon) used to determine whether a worker is within an acceptable work location.

**GPS Accuracy** The estimated horizontal error radius (in metres) reported by the device GPS receiver at the time of a location reading. A lower value indicates a more precise fix.

**JWT** JSON Web Token. A compact, URL-safe means of representing claims between two parties, used here for API authentication.

**Locale Preset** A named configuration bundle containing a default holiday list, timezone, and currency for a specific deployment region. The India preset is shipped with the system; others may be contributed by the community.

**NotificationProvider** The interface through which the backend dispatches push notifications to worker devices. Firebase Cloud Messaging is the default

implementation.

**Offline-First** An architectural pattern where all core application functions operate without internet connectivity, with data synced to the server when a connection becomes available.

**Paid Leave** An absent working day explicitly marked by an administrator as paid; the day counts toward the worker's wage calculation.

**Petty Cash** A small fund disbursed to workers for incidental work-related expenses. Managed via the petty-cash ledger feature.

**Required Work Duration** The minimum number of hours per day configured by the administrator that constitutes a full-attendance session. Sessions shorter than this trigger a short-hours warning.

**Short-Hours Flag** An indicator applied to an attendance record where the worker's session duration was less than the configured required work duration. Informational only; no automated penalty is applied.

**Sync** The process of transmitting locally stored records (attendance, petty-cash expenses, configuration updates) between the Android device and the backend server.

**TLS** Transport Layer Security. A cryptographic protocol providing secure communication over a network.

**Unpaid Leave** An absent working day classified (explicitly or by default) as unpaid; the day is excluded from the worker's wage calculation.

**UTC** Coordinated Universal Time. The primary time standard used for all timestamps in the system.

**Wage Strategy** A named algorithm used to calculate a worker's gross wages. Built-in strategies: `DAILY_FIXED`, `MONTHLY_DIVIDED`, `HOURLY`. Custom strategies may be registered by deployers.

**Weekly Off** The day(s) of the week on which a worker is not expected to attend. Configured per worker; defaults to Sunday.

**WorkManager** An Android Jetpack library for scheduling deferrable background tasks, used here for reliable sync and notification scheduling.

# B Issues List

The following open questions are pending resolution. Items marked with ⋆ are particularly relevant to contributors picking up work on the affected feature.

1. **Clock-Out Selfie:** Should the clock-out flow also require a selfie photograph, or is GPS alone sufficient?
2. **Edit Permissions:** Should both managers and admins be able to edit attendance records, or should this be restricted to admins only?
3. ⋆ **Poor GPS Fallback:** What is the intended behaviour when GPS accuracy is poor (e.g., 100 m error radius) but the worker is visibly on-site? Options: (a) reject and require retry, (b) allow with a flag, (c) allow manager override remotely.
4. **Half-Day Marking:** Is half-day attendance marking required in Version 1.0, or deferred to Phase 2? (See contribution roadmap, §1.4.)
5. **Camera Direction:** Should selfie capture use the front camera (true selfie) or the rear camera (higher quality)?
6. **Offline Login Window:** What is the maximum acceptable duration for cached offline login before re-authentication is enforced?
7. **Wage Precision:** Should rates support sub-unit decimal precision (e.g., paise for INR), or are whole-unit rates sufficient?
8. **Wage Visibility on App:** Should the worker's wage summary be visible at all times on the home screen, or only on a dedicated earnings screen (privacy consideration)?
9. **Holiday List Maintenance:** Who is responsible for updating the locale preset holiday list annually: the deploying organisation, or will the system provide an automated update mechanism?
10. **Multiple Weekly Off Days:** Can a worker have more than one weekly off day (e.g., Saturday and Sunday)? Is a half-day weekly off pattern required?
11. ⋆ **Petty-Cash Negative Balance:** Should the system allow a worker's petty-cash balance to go negative (i.e., can workers log expenses exceeding their current balance)?
12. ⋆ **Petty-Cash Offline Expense:** If a worker adds an expense offline and the balance would go negative upon sync, how should the conflict be resolved?
13. **Reminder Notification Time:** Is per-organisation configurability of reminder time sufficient, or should it also be configurable per worker?
14. **Short-Hours Threshold Uniformity:** Is the required work duration the same for all workers, or should it be configurable per worker or per role?
15. ⋆ **Multiple Geofence Zones:** Should a worker be permitted to clock in from any active geofence zone, or should zone assignment be per-worker?
16. ⋆ **Alternative Notification Providers:** Which providers (ntfy, Gotify, APNS,

SMTP) should be implemented for the v1.0 release, and which deferred to community contributions?

17. **Custom Wage Strategies:** What interface (plugin contract) should a deployer implement to register a custom wage strategy? This needs a design decision before FR-13 is implemented.